

Private and Constant Memory Handling For Automatic Kernel Fusion GPU Libraries

Oscar Amoros ¹ (oscar.amoros.huguet@upc.edu), Johnny Nuñez ³ (johnnunez@nvidia.com), Antonio J. Peña ^{1,2} (antonio.pena@bsc.es)

¹ Universitat Politècnica de Catalunya, ² Barcelona Supercomputing Center, ³ NVIDIA

Project repository: <https://github.com/Libraries-Openly-Fused>



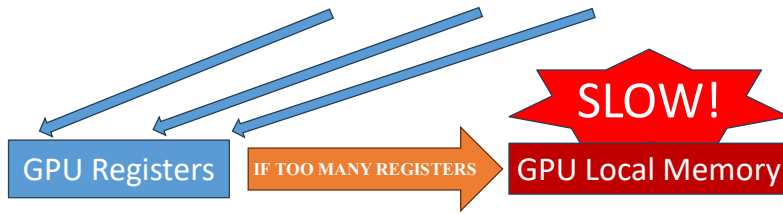
ABSTRACT



Kernel fusion and C++ template based kernel composition are very popular optimizations in GPU libraries, but when using big and complex kernel parameters, very often GPU registers get overcrowded and the compiler generates local memory transfers, which kill the kernel performance. The Fused Kernel Library (FKL) which implements automatic kernel fusion based on C++17/20 template metaprogramming, is a clear case where very complex structs, which can include static arrays and tuples, are passed as parameters to the GPU Kernel. In this poster, we explain why this issues happen and provide a very simple set of rules to completely avoid those issues.

BACKGROUND

```
__global__ void myKernel(Type1 param1, Type2 param2, Type3 param3... ) {
```

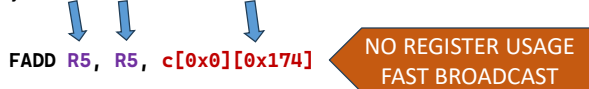


```
__device__ void function(Type1* param1, Type2& param2, ...) {  
    int i = // compute index at runtime  
    param1[i] += param2[i];  
}  
  
Type1 param1[10]; Type2 param2[10]; // Static arrays  
function(param1, param2, ...);
```

CONTRIBUTION: Fused Kernel Library (FKL)

SOLUTION GRID CONSTANT AND CONST &

```
__global__ void myKernel(__grid_constant__ const Type1 param1,...){  
__device__ float function(const float val, const Type1& param1) {  
    int i = // compute index at runtime  
    // Runtime computed constant memory address  
    param1.image[i].data[threadIdx.x ...] = - ;  
    // constant memory access in the instruction  
    return val + param1.mulVal ;  
}  
}
```



FKL OPERATIONS AND INSTANTIABLE OPERATIONS

Instantiable Operation

```
.then(NextIOp)  
operator&(NextIOp)
```

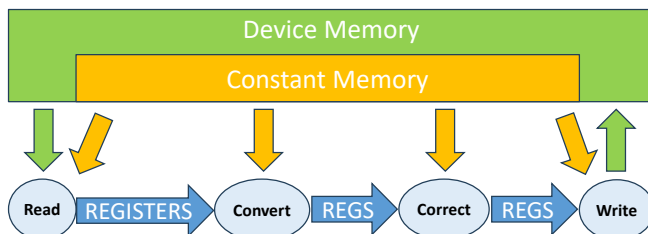
```
OPERATION  
Output exec(input, const OperationData&)  
InstantiableOpType build(...)
```

then() and operator&() -> Create a new IOp containing both.

exec() -> Implementation of the Operation

build() -> Converts user friendly arguments into kernel friendly arguments. Returns an Instantiable Operation that stores the computed kernel friendly arguments.

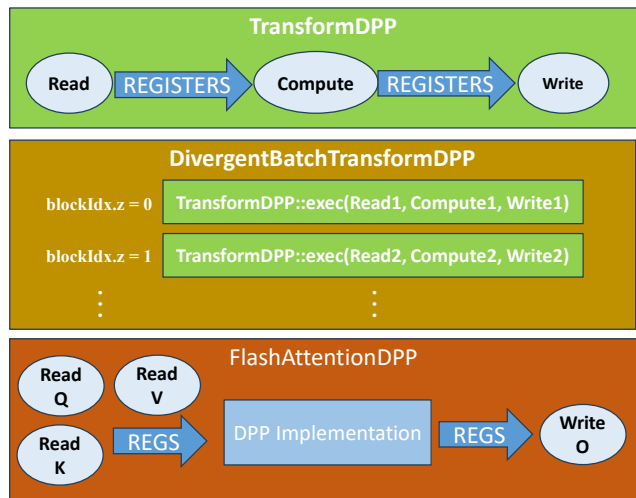
```
auto readIOp = ReadYUV::build(yuvImage);  
auto computeOp =  
ConvertYUVToRGB::build(colorDetails).then(ColorCorrect::build(colCorrDet));  
auto writeOp = Write::build(rgbImage);  
  
executeOperations<TransformDPP>(stream, readIOp, computeOp, writeOp);
```



SUPPORTS VERY COMPLEX KERNEL PARAMETERS

```
Array<Rect, 50> crops{...};  
executeOperations<TransformDPP>(stream, readNV12, yuvToRGB, crops,  
resize, addBorder, mul, sub, div, tensorWrite);  
  
// Internally we fuse until the last Read Operation  
auto readOperation = readNV12 & yuvToRGB & crops & resize & addBorder;  
BatchRead<50, AddBorder<Resize<Crop<F0p<ReadNV12, YUVtoRGB>>>>>
```

FKL Data Parallel Patterns



RESULTS, CONCLUSIONS AND FUTURE WORK

No local memory accesses added due to IOps with very complex kernel parameters.

The number of registers per thread that an additional Operation adds to the kernel, depends (mostly*) on the exec() implementation, not on the complexity of the parameters.

Actively used in a real world application, AutomaticTV at Mediapro.

Image processing speedups proportional to the number of kernels being fused, from 2x to 200x. The bigger the GPU, the greater the speedups.

We can forget about the kernel parameter problem and focus on other aspects of the algorithm implementation.

Currently working on adapting FlashAttention (v2) to FKL philosophy.